

Tor Transport — Deep Dive

Companion to [ARCHITECTURE.md](#). Covers what Tor actually buys you, the three possible topologies, onion service mechanics, the concrete iOS implementation path (including one trap that will cost you a day), and what remains exposed afterwards.

1. The headline: your design makes the relay optional

Store-and-forward messengers **need** a server, because the recipient is by definition absent. Yours doesn't. You have already committed to both parties being online simultaneously — and *that is exactly the precondition that makes peer-to-peer onion services work*.

This is worth sitting with. The synchronous constraint, which looks like a product limitation, is what unlocks the strongest architecture available: **each client runs its own onion service and peers connect directly**. No relay, no operator, no third party to compel, no central point that observes who talks to whom. This is Ricochet Refresh's model, and it is the genuine version of the "serverless" idea from your original framing.

The relay becomes a scaffold you can remove — or shrink to a pure rendezvous signal that never sees message traffic (§10).

2. What Tor fixes, and what it doesn't

Exposure	Without Tor	With Tor
Client IP → relay	Fully visible	Removed
Relay IP → adversary	Visible; DDoS-able, seizable, blockable	Removed (onion service)
Who talks to whom	Relay sees the pairing	Removed only in P2P topology
When they talk	Precise	Still precise
Message sizes	Visible	Still visible — pad

Exposure	Without Tor	With Tor
Message frequency / burstiness	Visible	Still visible — pad or cover traffic
Content	Protected by E2E	Protected by E2E
The out-of-band invite channel	Exposed	Still exposed

Tor is an IP-address anonymiser, not a traffic-analysis defence. Timing and volume survive it. Under synchronous-only messaging, timing is your loudest signal — so Tor is necessary but not sufficient, and §9 matters as much as this section.

Note also that Tor removes the need for a CA. A v3 onion address **is** a public key, so the address self-authenticates. No Let's Encrypt, no certificate pinning, no CA to compromise.

3. Three topologies

(a) Client → Tor → clearnet relay

[iOS] — Tor circuit —► [exit node] — TLS —► [relay: chat.example.com]

Relay no longer learns client IPs. But an exit node is involved (sees destination hostname via SNI), the relay's own IP is public and blockable, and you need real TLS and a real CA.

Verdict: stepping stone at best. Weakest option.

(b) Relay as an onion service

[iOS] — 3 hops —► [rendezvous] ◄— 3 hops — [relay: xyz...onion]

No exit node. Relay IP hidden — it can run anywhere, including a machine with no inbound ports. Address self-authenticating. Traffic stays inside the Tor network end to end.

Verdict: the right Phase-3 target. Large gain, low complexity.

(c) Peer-to-peer onion services — no relay

[A: abc...onion] ◄— 6 hops via rendezvous —► [B: def...onion]

Each client runs an onion service; the "room ID" is replaced by the peer's onion address. Nobody in the middle. Nothing to subpoena, because nothing exists.

Verdict: the endgame, and reachable **because** of the synchronous design. Costs: an embedded Tor daemon on both devices, ~5–30s bootstrap, foreground-only operation, and the rendezvous problem (§10).

Recommended path: (b) at Phase 3, (c) at Phase 4. Both parties already run Tor in (b), so (c) is mostly a matter of publishing a descriptor rather than a new subsystem.

4. Onion service v3 mechanics

Worth understanding, because two properties are directly useful to you.

The address is the key.

```
onion_address = base32( ed25519_pubkey || checksum || version ) + ".onion" // 56 chars
```

Connecting to **abc...onion** cryptographically guarantees you reached the holder of that private key. No MITM is possible at the transport layer, from anyone, ever. (Keep the app-layer safety-number check anyway — it authenticates the *human*, and defends against a compromised device having published a substitute address.)

How a connection forms:

1. Service picks introduction points, publishes a signed descriptor to the hash ring.
2. Client fetches the descriptor from a directory (using a blinded key derived from the address — directories learn nothing about which service is being looked up).
3. Client picks a rendezvous point, tells the service via an introduction point.
4. Both build circuits to the rendezvous point. Three hops each: **six hops total**.

Descriptor blinding means the directory servers cannot enumerate services or tell who is being resolved. Combined with client auth (§5), an unauthorised party cannot even confirm your service exists.

5. Client authorisation — this is your invite mechanism

v3 client auth encrypts the service descriptor to specific x25519 client keys. Without the key, a client cannot fetch a usable descriptor, cannot find the introduction points, and **cannot even determine the service exists**.

This maps onto your model exactly. Your original design had a room UUID that was an unrevokable bearer capability: anyone who learned it had permanent access. Replace it:

invite = onion_address + client_auth_private_key + identity_fingerprint

You now get, at the network layer and for free:

- Unauthorised parties cannot connect, probe, or confirm existence.
- Access is **revocable** — remove the client's key from the service and re-publish.
- Presence probing by a leaked-ID holder becomes impossible.
- Scanning and enumeration attacks are eliminated.

Server side (`torrc`):

HiddenServiceDir /var/lib/tor/chat/

HiddenServicePort 80 127.0.0.1:8080

HiddenServiceVersion 3

Then per-client, in `/var/lib/tor/chat/authorized_clients/alice.auth`:

descriptor:x25519:<base32-encoded client public key>

Client side, the matching private key goes in `ClientOnionAuthDir`. In an embedded iOS Tor this is a file you write before starting the daemon.

Bind the relay to `127.0.0.1`, never `0.0.0.0`. Otherwise the service is reachable over clearnet too and the whole exercise is undone by a port scan.

6. iOS: getting Tor into the app

Option	Notes
Tor.framework (Guardian Project / Onion Browser)	Embeds the C tor daemon. Battle-tested — it is what Onion Browser ships. CocoaPods/SPM. Recommended.
Arti (Rust rewrite)	Cleaner to embed, better API surface, actively developed. Onion-service support has been landing progressively — verify current maturity for <i>service</i> hosting, not just client use, before committing. Worth re-checking; this space moves.
Orbot iOS (system VPN)	Requires users to install a separate app; you don't control lifecycle. Not viable for a product.

App Store distribution of Tor-embedding apps is established practice (Onion Browser has shipped for years), but expect review questions. Have an export-compliance answer ready.

Lifecycle

Tor bootstrap is **5–30 seconds** on mobile networks, occasionally worse. Design for it:

- Start bootstrap eagerly when the app enters foreground, before the user asks to connect.
 - Surface bootstrap progress in the UI — Tor emits percentage events, so show them. A 20-second unexplained spinner reads as a broken app.
 - iOS suspends background sockets aggressively; assume the daemon dies when backgrounded and plan for full re-bootstrap on return.
 - Persisting Tor's state directory across launches speeds up subsequent bootstraps (cached consensus, guard selection) — and keeping stable **guard nodes** is a genuine security property, not just an optimisation. Guard rotation increases exposure to a malicious first hop.
-

7. The trap: WebSocket over SOCKS5 on iOS

URLSessionWebSocketTask cannot reliably be routed through a **SOCKS5** proxy on iOS. **connectionProxyDictionary** supports SOCKS on macOS; the iOS behaviour is undocumented and unreliable, and — worse — it can *silently fall through to the direct connection*, which deanonymises the user while appearing to work. Do not use it. Do not test it once, see traffic flow, and conclude it works.

Use `Network.framework` instead. iOS 17+ has first-class SOCKSv5 support:

```
let proxy = ProxyConfiguration(socksv5Proxy: .hostPort(host: "127.0.0.1", port: 9050))

let params = NWParameters.tcp

params.proxyConfiguration = proxy          // NWParameters.PrivacyContext on some paths

let ws = NWProtocolWebSocket.Options()

ws.autoReplyPing = true

params.defaultProtocolStack.applicationProtocols.insert(ws, at: 0)

let conn = NWConnection(

    host: .name("abc...onion", nil),        // ← name, NOT a resolved address

    port: 80,

    using: params

)
```

Pre-iOS-17, or if you need more control: open a raw **NWConnection**, perform the SOCKS5 handshake yourself (it is a short, well-specified exchange), then run a WebSocket library over the resulting stream. **CocoaAsyncSocket** or a Starscream custom-transport shim both work.

Critical: never resolve `.onion` locally

`.onion` names have no DNS existence. They **must** be passed to the SOCKS proxy as a domain-type address (SOCKS5 ATYP 0x03) so Tor resolves them internally. Any code path that calls **getaddrinfo**, or that resolves a hostname before connecting, leaks a DNS query revealing that the user is contacting a hidden service — and then fails anyway.

Audit for: `NWEndpoint.hostPort` with a resolved IP, any `.address` variant, URL loading that resolves first, and analytics or crash SDKs that resolve hostnames independently.

Add a leak test to CI. Run the app against a network namespace with no direct route and assert that zero packets leave outside the Tor process. "It seemed to work" is not verification for this class of bug.

No TLS inside the onion

The onion protocol already provides authentication and encryption to the endpoint. `ws://` inside a v3 onion service is fine. Adding TLS gains little and forces you back into CA questions. Your payloads are E2E-encrypted regardless.

8. Performance budget

Metric	Cleartnet	Onion (topology b)	P2P onion (c)
Hops	0	6	6
Typical RTT	20–80 ms	200–800 ms	300–1000 ms
Bootstrap	—	5–30 s	5–30 s, both sides
Throughput	Link speed	Often < 1 Mbit/s	Often < 1 Mbit/s
Battery	Baseline	Noticeably higher	Higher still

Implications:

- Text chat is comfortable. **Real-time indicators are not** — which conveniently agrees with §9's recommendation to remove them for security reasons.
 - Media transfer needs chunking, resumability, and honest progress UI.
 - Set connection timeouts in the tens of seconds, not the single digits. Aggressive timeouts over Tor produce phantom failures and reconnect storms.
 - Expect circuits to die spontaneously. Reconnection is a normal event, not an error state — but per `ARCHITECTURE.md` §6.5 it ends the session and requires a fresh handshake, so make that visible rather than papering over it.
-

9. Residual metadata and what to do about it

Tor removes IPs. It does not remove these:

Message length. Ciphertext length tracks plaintext length. Pad to fixed buckets — e.g. 256 / 1024 / 4096 bytes — inside the encrypted envelope, before encryption. Cheap, and it defeats naive length-based inference.

Timing. The strongest remaining signal, and the one your design amplifies. Both parties online in the same window is itself the metadata. In topology (c) nobody is positioned to observe the pairing, which is the real argument for getting there.

Keystroke timing. Typing indicators transmit inter-keystroke intervals, which are a well-studied channel for inferring what is being typed. **Remove typing indicators.** They are also useless over a 500ms circuit. Same reasoning applies to per-keystroke anything.

Cover traffic. The thorough answer is fixed-rate padding — send a constant stream of same-size frames whether or not there is a message. It genuinely defeats timing analysis. It also costs battery and bandwidth continuously. Reasonable compromise: pad within an active session only, where the marginal cost is bounded and the user has opted into a session anyway.

Traffic to the introduction/rendezvous infrastructure reveals that *someone* is running a hidden service, though client auth prevents identifying which.

10. The rendezvous problem

This is the hardest unsolved problem in the design, and it deserves to be stated plainly rather than deferred.

Both parties must be online simultaneously. But iOS suspends backgrounded apps, so an onion service on a phone only exists while the app is in the foreground. **So how does B know to open the app?**

The obvious answer — push notifications — requires APNs, which requires a server that knows a device token, and every push reveals to Apple and to your notification server that *this device is being contacted now*. That reintroduces exactly the metadata you removed. Note that this is a problem for Signal too; they accept it. You may not want to.

Options, none free:

Approach	Cost
Scheduled sessions — agree a window out of band	No infrastructure, no metadata. Poor UX, but honest, and arguably correct for the actual threat model.
Minimal rendezvous relay — a signal-only service that knows "someone is waiting at room R", never sees traffic	Keeps topology (c) for content; small metadata surface remains. Pragmatic middle ground.
APNs with an opaque token — push carries no content, only "open the app"	Best UX. Apple learns contact timing. Push token becomes a persistent identifier — a real cost.
Foreground polling — app checks for waiting peers while open	No push infrastructure, but only works if the app is already open, which is the problem you were solving.

Recommendation: ship scheduled/manual sessions first. It is honest about the model, requires nothing, and lets you validate whether users accept the constraint — which is the actual product risk. Add a signal-only rendezvous relay if they don't. Treat APNs as an explicit, documented, user-visible tradeoff, never a silent default.

This is also the decision that determines whether the relay in `relay-server.mjs` disappears entirely or shrinks into a rendezvous beacon. Everything else in the architecture is settled; this one isn't.

11. Phasing

Phase	Work	Delivers
3a	Relay as onion service; <code>torrc</code> + client auth files	Relay IP hidden, no exit nodes, invites become revocable
3b	Embed Tor.framework; <code>NWConnection</code> + SOCKS5; leak tests in CI	Client IPs hidden from relay
3c	Padding buckets; remove typing indicators	Length and keystroke channels closed

Phase	Work	Delivers
4a	Client-hosted onion services; peer address replaces room UUID	Relay leaves the trust graph
4b	Rendezvous decision (§10)	Determines whether any server survives

Do not attempt Tor before the plaintext-transport protocol is stable and tested. Debugging a signature-verification bug through a six-hop circuit with a 30-second bootstrap is a bad afternoon, and every failure looks identical to a network failure.